

Triton

GPU Kernel Programming in Python — 14 Sessions + Final Project

Vector ops · Softmax · LayerNorm · GEMM · FlashAttention · Autotuning · PyTorch integration · 2026 Edition · Updated June 04, 2026

4

Phases

14

Sessions

1

Final Project

~6

Weeks to Complete

Why Triton in 2026: Triton is now the primary language for writing new ML kernels. FlashAttention 2 and 3, Torch Inductor, vLLM attention ops, and most production inference kernels are written in Triton. Raw CUDA still exists for micro-optimization, but if you can write a correct, autotuned Triton kernel — you can contribute to any major ML inference project. **Prerequisite:** Complete CUDA Course Phases 1-2 first. Triton abstracts thread management but not memory hierarchy — you need that mental model.

Triton vs CUDA — Know When to Use Which

Dimension	Raw CUDA	Triton
Abstraction level	Thread-level — you manage every thread	Block-level — compiler handles thread scheduling
Memory management	Manual: <code>__shared__</code> , <code>__syncthreads</code> , coalescing	Automatic: compiler infers shared mem usage and sync
Language	C with GPU extensions, compiled with <code>nvcc</code>	Python with <code>@triton.jit</code> decorator, JIT compiled
When to use	Micro-optimization, custom hardware features, ultra-low latency	New ML kernels — 90% of use cases in 2026
Learning curve	High — need full CUDA Course first	Medium — need CUDA Phase 1-2 conceptually, Triton handles the rest
Debugging	<code>cuda-gdb</code> , <code>printf</code> in kernel, Nsight	<code>TRITON_INTERPRET=1</code> for CPU debug, Nsight for profiling PTX

P1

TRITON FUNDAMENTALS

Sessions 1–4 · Mental model, first kernel, masks, reductions · ~4 days

#	Topic	Key File	What You Learn / Build	Time
0 1	Triton Mental Model	triton_intro.py	How Triton differs from CUDA: programs run at the block level (not thread level), <code>tl.program_id()</code> gives block index, Triton handles thread scheduling inside the block automatically. The compiler does what you'd do manually in CUDA: vectorization, memory coalescing, shared mem tiling. Prerequisites: complete CUDA Course Phase 1 first — Triton abstracts CUDA, not replaces the knowledge.	1 day
0 2	First Triton Kernel	vector_add.py	<code>@triton.jit</code> decorator, <code>tl.program_id(axis=0)</code> , <code>offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)</code> , <code>tl.load(ptr + offsets, mask=mask)</code> , <code>tl.store</code> . Compare to your CUDA <code>vector_add.cu</code> — same logic, 3x less code. Run with <code>triton_kernels.vector_add[grid](x, y, z, n, BLOCK_SIZE=1024)</code> .	1 day
0 3	Memory Access + Masks	masked_load.py	Mask pattern: <code>mask = offsets < n</code> — prevents out-of-bounds reads at tensor boundaries. <code>tl.load</code> with <code>other=0.0</code> as fill for masked lanes. Block pointer API (Triton 2.2+): <code>tl.make_block_ptr</code> for 2D tensors — cleaner than manual offset math. Understanding when Triton generates coalesced vs strided memory instructions.	1 day
0 4	Reductions	reduce.py	<code>tl.sum(x, axis=0)</code> , <code>tl.max</code> , <code>tl.min</code> — warp-level reductions handled automatically by Triton compiler. Row-wise sum: each program handles one row, reduce across <code>BLOCK_SIZE</code> columns. Compare to your <code>warp_reduce.cu</code> — Triton does in 5 lines what took 30 in CUDA. Understand what PTX Triton generates via <code>triton.compiler.compile()</code> .	1 day

P2

CORE ML KERNELS

Sessions 5–8 · Fused ops, softmax, LayerNorm/RMSNorm, autotuning · ~6 days

#	Topic	Key File	What You Learn / Build	Time
0 5	Fused Elementwise Ops	fused_ops.py	Fuse multiple elementwise ops into one kernel: <code>load</code> → <code>ReLU</code> → <code>multiply by scalar</code> → <code>store</code> . Why fusion beats separate kernels: each kernel launch reads/writes global memory. Fused: one read + one write. Unfused: N reads + N writes. Benchmark: fused vs torch elementwise chain — measure HBM bandwidth utilization.	1 day
0 6	Softmax Kernel	softmax.py	Safe softmax: subtract row max before exp (numerical stability). Two-pass: one pass for max, one for sum. Single-pass online softmax: track running max + running sum simultaneously (Milakov & Gimelshein algorithm). Online softmax is the core insight behind FlashAttention — master it here. Benchmark vs <code>torch.softmax</code> : match or exceed on sequence lengths > 1024.	2 days
0 7	LayerNorm + RMSNorm	layer_norm.py	LayerNorm: mean + variance in one pass (Welford), normalize, scale + bias. RMSNorm: simpler — only RMS (no mean subtraction), used in LLaMA/Mistral/Gemma. Fused forward+backward: store intermediate activations for gradient computation. Understand why RMSNorm is faster (one less reduction) and equally effective.	2 days

#	Topic	Key File	What You Learn / Build	Time
08	Autotuning	autotuned_kernel.py	@triton.autotune(configs=[...], key=['n_rows', 'n_cols']) — Triton searches over BLOCK_SIZE_M, BLOCK_SIZE_N, BLOCK_SIZE_K, num_stages, num_warps configs automatically. Configs: list of triton.Config({...}). Key: tensor dimensions that affect optimal config. How Triton caches autotune results (JSON file). When to autotune vs hardcoded.	1 day

P3

MATRIX OPS + FLASHATTENTION

Sessions 9–12 · GEMM, fused dropout, FlashAttention fwd + bwd · ~10 days

#	Topic	Key File	What You Learn / Build	Time
09	GEMM in Triton	matmul.py	Tiled matrix multiply: load A-tile [M, K] and B-tile [K, N] into SRAM, tl.dot(a, b) for tensor core computation, accumulate in fp32, store fp16. K-loop tiling: iterate over K dimension in BLOCK_K chunks. This is the reference implementation — study how it compares to cuBLAS at different shapes. num_stages for software pipelining (async prefetch while computing).	3 days
10	Fused Dropout + GELU	fused_dropout.py	Stochastic ops in Triton: tl.rand(seed, offsets) for per-element random. Fused dropout + GELU: load → apply GELU → apply dropout mask → store. Philox RNG in Triton: offset-based generation ensures reproducibility with same seed. Benchmark: fused vs two separate PyTorch ops on forward pass memory bandwidth.	1 day
11	FlashAttention Forward	flash_attn_fwd.py	Tiled attention: split Q/K/V into blocks, compute attention scores block by block. Online softmax running across K-blocks: maintain (max, sum) state per Q-row. Never materialize the full N×N attention matrix — O(N) memory instead of O(N²). Causal masking: tl.where(col_idx > row_idx, float('-inf'), scores). Verify output matches F.scaled_dot_product_attention to 1e-3 tolerance.	3 days
12	FlashAttention Backward	flash_attn_bwd.py	Recomputation: re-run forward on each block during backward instead of storing attention weights. Gradient for Q, K, V via chain rule through the tiled online-softmax forward. $dV = P^T dO$, $dK = P^T dQ$, $dQ = P dK$ (with softmax Jacobian). This is the hardest session — read the FA2 paper alongside the code.	3 days

Sessions 11-12 are the hardest in the course — expect to spend real time here.

FlashAttention backward requires understanding the chain rule through online softmax. Read the FlashAttention 2 paper (Dao 2023) alongside Session 12. If the backward pass isn't clicking, implement a naive backward first in PyTorch, verify gradients with torch.autograd.gradcheck, then port to Triton.

P4

PRODUCTION + DEBUGGING

Sessions 13–14 · PyTorch integration, autocast, debugging, Nsight · ~4 days

#	Topic	Key File	What You Learn / Build	Time
1 3	PyTorch Integration	attention_op.py	torch.autograd.Function with @staticmethod forward/backward calling Triton kernels. TORCH_LIBRARY + TORCH_LIBRARY_IMPL for TorchScript + torch.compile compatibility. torch.compile + Triton: @torch.compile will call your Triton kernel directly. Handling autocast (bf16/fp16): check input.dtype and dispatch correct kernel config.	2 days
1 4	Debugging + Benchmarking	bench_debug.py	Triton interpreter mode: TRITON_INTERPRET=1 python script.py — runs on CPU, enables pdb breakpoints. triton.testing.do_bench(lambda: kernel[grid](...)) — measures median + p99 kernel time. Nsight Compute with Triton: ncu --set full python script.py — profile generated PTX. Common bugs: wrong mask shape, missing tl.multiple_of hint (breaks vectorization), num_stages too high (exceeds shared memory). Reading Triton PTX output.	2 days

Final Project — Production Fused MHA + KV Cache

Full FlashAttention fwd + bwd in Triton, KV cache kernels, PyTorch op wrapper, benchmark table with real numbers. This is the project that opens doors at vLLM, CoreWeave, Baseten.

FINAL PROJECT	Production Fused MHA + KV Cache		FlashAttention + KV cache + PyTorch op + benchmarks		→ CAPSTONE
Skills covered:	C Triton	B FlashAttn	S PyTorch	B KV Cache	B Profiling
Build in phases: Phase 1: FlashAttention Forward — Tiled attention, online softmax, causal mask, verify vs <code>F.scaled_dot_product_attention</code> Phase 2: FlashAttention Backward — Recomputation-based gradient, verify gradients vs autograd, handle GQA Phase 3: PyTorch Op + Autocast — <code>torch.autograd.Function</code> wrapper, bf16/fp16 dispatch, <code>torch.compile</code> compatible Phase 4: KV Cache + Benchmark — Add KV cache write/read kernels, benchmark vs PyTorch at seq_len 512/1024/4096/8192 Key files: ■ <code>flash_attn/fwd.py</code> — Triton FlashAttention forward with causal + non-causal ■ <code>flash_attn/bwd.py</code> — Triton FlashAttention backward with recomputation ■ <code>flash_attn/interface.py</code> — <code>torch.autograd.Function</code> + autocast dispatch ■ <code>kv_cache/cache_ops.py</code> — Triton KV cache write + paged gather kernels ■ <code>bench/run_bench.py</code> — <code>triton.testing.do_bench</code> at multiple seq_len/batch combos ■ <code>bench/memory_bench.py</code> — peak memory usage vs naive attention			Put this on your resume: ◆ <i>Implemented FlashAttention forward + backward in Triton: $O(N)$ memory vs $O(N^2)$ naive; 2.3x throughput vs PyTorch SDPA at seq_len=4096, BF16 on A100</i> ◆ <i>Wrapped as <code>torch.autograd.Function</code> with bf16/fp16 autocast dispatch; output matches <code>F.scaled_dot_product_attention</code> to 1e-3; <code>torch.compile</code> compatible</i> ◆ <i>Added Triton KV cache kernels for paged attention — matched vLLM reference implementation in per-token decode latency benchmark</i>		

JD Coverage — Why Triton Matters for Your Job Search

Triton maps directly to the A-tier inference optimization skill (24/40 JDs). These are the exact role types where Triton proficiency is the differentiator.

Inference Optimization (A-tier, 24/40 JDs) — Triton IS how production inference kernels are written.

vLLM, SGLang, LightLLM, and TGI all use Triton kernels for attention, layernorm, and activation ops. Baseten and CoreWeave explicitly hire for Triton kernel development. FlashAttention 2 and 3 are written in Triton. If you can write and optimize Triton kernels, you can contribute to any of these projects on day 1.

GPU Infrastructure (A-tier, 27/40 JDs) — Triton knowledge signals deep GPU understanding.

Writing a correct Triton GEMM or softmax kernel demonstrates you understand memory hierarchy, occupancy, and bandwidth limitations — the same concepts GPU infra engineers debug in clusters. Companies hiring for GPU infra use Triton proficiency as a proxy for GPU architecture knowledge.

PyTorch custom ops (Sessions 13) — required for ML infrastructure roles at AI labs.

Physical Intelligence, Anthropic, Character.AI, and Decagon all have custom kernel requirements. torch.compile integration (Session 13) is increasingly required — Torch Inductor generates Triton kernels and knowing how to override them with custom implementations is a 2026 skill.

Agentic AI / LLM deployment — Triton enables the inference speed that makes production LLMs viable.

PagedAttention (vLLM), continuous batching, speculative decoding — all built on Triton attention kernels. Understanding FlashAttention mechanics (Sessions 11-12) is what lets you tune vLLM for your hardware rather than just running the defaults.

Session Timeline — 6 Weeks Start to Finish

Phases 1–4 in ~24 days. Final project in 3 weeks.

When	Phase	Deliverable
Days 1–4	Phase 1	Mental model, vector add, masked loads, reductions — Triton basics running
Days 5–10	Phase 2	Fused ops, softmax, LayerNorm/RMSNorm, autotuning — core ML kernels
Days 11–20	Phase 3	GEMM, fused dropout, FlashAttention fwd + bwd — the hard sessions
Days 21–24	Phase 4	PyTorch integration, autocast, debugging, Nsight profiling
Wk 4	Project P1	FA forward + correctness verification vs F.scaled_dot_product_attention
Wk 5	Project P2	FA backward + PyTorch op wrapper with autocast
Wk 6	Project P3	KV cache kernels + benchmark table at multiple seq_len — README with numbers

What NOT to Do

Mistake	Why
Learning Triton without CUDA basics	BLOCK_SIZE, tiling, and reductions make no sense without the CUDA mental model. Do CUDA P1-P2 first.
Triton for simple elementwise ops	torch.compile already generates good Triton for elementwise. Write custom Triton only for fused or complex kernels.
Rewriting cuBLAS in Triton	cuBLAS is years of expert tuning. Match it at specific shapes, don't try to beat it universally.
TRITON_INTERPRET for performance testing	Interpreter runs on CPU — 1000x slower. Only use for correctness debugging, never for benchmarking.
Old Triton tutorials (pre 2.0)	Triton 1.x had different APIs. Block pointer syntax, triton.Config, and autotune changed significantly in 2.x.
Writing Triton before verifying correctness	Always compare output to torch reference (torch.allclose) before benchmarking. Silent numerical errors are common.

Do CUDA Course Phases 1-2 before starting this. After the final project: read the FlashAttention 3 paper and study the Triton source for vLLM's paged attention kernel.